

Pràctica 3: Codificació en SAT

Lògica en la Informàtica

FIB

Antoni Lozano

Q1 2025–2026

Objectius

Aquesta pràctica té com a objectius:

- Generar la codificació de problemes en SAT mitjançant un programa en Prolog
- Aprendre a utilitzar la llibreria de Prolog que es proporciona com a ajut per la codificació
- Fer ús d'un *SAT solver* (**kissat**) per resoldre problemes combinatoris

Referències

Com a guia d'estudi teniu

- El fitxer **miSudoku.pl** i els d'entrada, que serveixen d'exemple per la resta de problemes
- El **Makefile** que es proporciona

Codificació en SAT

1 Generació de clàusules

2 Predicats útils

3 Codificació de sudokus

4 Llibreria adjunta

5 Exercicis

Codificació en SAT

El programa

- 1 genera i desa en un fitxer `infile.cnf` les clàusules corresponents a les restriccions del problema
- 2 crida al *SAT solver* `kissat` amb `infile.cnf` i desa el resultat en un fitxer `model`
- 3 obté un model simbòlic M , que conté les variables certes de la solució en format simbòlic
- 4 mostra la solució de forma llegible amb `displaySol`

Codificació en SAT

Per codificar un problema, només cal modificar tres punts de l'esquema:

- 1 Definir les *SAT variables*, que poden venir donades totalment o parcial
- 2 Generar les clàusules corresponents a les restriccions, és a dir, completar el predicat `writeClauses/0`
(No confondre amb `writeOneClause/1`, que afegeix una clàusula)
- 3 Escriure o completar `displaySol(M)`, on `M` és la llista de variables simbòliques (*SAT variables*) que són certes en el model retornat per `kissat`

Codificació en SAT

1 Generació de clàusules

2 Predicats útils

3 Codificació de sudokus

4 Llibreria adjunta

5 Exercicis

Predicat between/3

El predicat **between**(L,U,X) és cert quan X és un enter entre L i U.

```
?- between(1,4,2) .  
true.
```

```
?- between(1,4,X), write(X), nl, fail.  
1  
2  
3  
4  
false.
```


Predicat `findall/3`

El predicat **`findall`**(X, G, L) és cert quan L és la llista dels X per als quals l'objectiu G és cert (instanciant les variables necessàries).

`f(a,b,c).`

`f(a,b,d).`

`f(b,c,e).`

`f(b,c,f).`

?- **`findall`**($C, f(A, B, C), L$).

$L = [c, d, e, f]$.

?- **`findall`**($C, f(A, c, C), L$).

$L = [e, f]$.

?- **`findall`**($C, f(A, a, C), L$).

$L = []$.

?- **`findall`**($A, f(A, b, C), L$).

$L = [a, a]$.

Predicat `findall/3`

El predicat **`findall`**(X, G, L) és cert quan L és la llista dels X per als quals l'objectiu G és cert (instanciant les variables necessàries).

`f(a,b,c).`

`f(a,b,d).`

`f(b,c,e).`

`f(b,c,f).`

?- **`findall`**($A, f(A,b,C), L$).

$L = [a,a].$

Per evitar obtenir una llista amb repeticions, es pot fer servir `sort/2`, que ordena i **elimina repeticions**.

?- **`findall`**($A, f(A,b,C), L$), `sort`(L, M).

$L = [a,a],$

$M = [a].$

Predicat `findall/3`

El predicat **`findall/3`** també admet condicions múltiples:

% llista dels nombres senars entre 1 i 7

?- **`findall(X, (between(1,7,X), 1 is X mod 2), L)`**.

`L = [1,3,5,7]`.

% llista dels quadrats dels senars entre 1 i 7

?- **`findall(Y, (between(1,7,X), 1 is X mod 2, Y is X*X), L)`**.

`L = [1,9,25,49]`.

Predicat nth1/3

El predicat **nth1**(N,L,E) és cert quan l'N-èsim element de L és E.

% cerca del segon element d'una llista

?- **nth1**(2, [15,42,55,79], E).

E = 42.

% cerca de l'element de la fila 2, columna 1 d'una matriu 2x2

?- **nth1**(2, [[4,2],[7,5]], F), **nth1**(1, F, E).

F = [7, 5],

E = 7.

Codificació en SAT

1 Generació de clàusules

2 Predicats útils

3 Codificació de sudokus

4 Llibreria adjunta

5 Exercicis

Sudoku

```
symbolicOutput(0).% set to 1 for DEBUGGING: to see symbolic output only; 0 otherwise.
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% A sudoku is a logic-based, combinatorial number-placement puzzle that
%% uses a partially filled 9x9 grid. The objective is to fill the grid
%% with the digits 1 to 9, so that each column, each row, and each of the
%% nine 3x3 blocks contain only one of each digit.
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

llista de llistes

solució única

```
%%%%%%%% begin input example sudoku1 %%%%%%%%%
%%
%% entrada([[-,4,-,-,-,-,1,-], % Solution shown ..... 6 4 3 9 7 5 2 1 8 .....
%%          [-,-,8,-,3,-,9,-,-], % for this ..... 2 7 8 4 3 1 9 5 6 .....
%%          [-,-,-,6,8,2,-,-,-], % input example: ..... 5 1 9 6 8 2 3 4 7 .....
%%          .....
%%          [3,2,-,-,6,-,-,7,9], % ..... 3 2 5 8 6 4 1 7 9 .....
%%          [-,-,7,-,-,-,4,-,-], % ..... 1 8 7 3 5 9 4 6 2 .....
%%          [9,6,-,-,1,-,-,8,3], % ..... 9 6 4 2 1 7 5 8 3 .....
%%          .....
%%          [-,-,-,7,9,8,-,-,-], % ..... 4 5 2 7 9 8 6 3 1 .....
%%          [-,-,1,-,2,-,7,-,-], % ..... 8 3 1 5 2 6 7 9 4 .....
%%          [-,9,-,-,-,-,2,-]]) % ..... 7 9 6 1 4 3 8 2 5 .....
%%
%%%%%%%% end input example sudoku1 %%%%%%%%%
```

Sudoku

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
% miSudoku.pl: simple example of our LI Prolog method for solving problems using a SAT solver.
%
% It generates the SAT clauses, calls the SAT solver, and shows the solution. Just specify:
% ..... 1. SAT Variables
% ..... 2. Clause generation
% ..... 3. DisplaySol: show the solution.
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Some helpful definitions to make the code cleaner: =====

row(I) :- between(1,9,I).
col(J) :- between(1,9,J).
val(K) :- between(1,9,K).

blockID(Iid,Jid) :- member(Iid,[0,1,2]), member(Jid,[0,1,2]). %there are 9 blocks: 0-0 1-0 ... 2-2
squareOfBlock(Iid,Jid,I,J) :- row(I), col(J), Iid is (I-1)//3, Jid is (J-1)//3.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% End helpful definitions =====

```

Sudoku

%%%%%%%% 1. SAT Variables: =====

% $x(I,J,K)$ means "square IJ gets value K ", $1 \leq i \leq 9$, $1 \leq j \leq 9$, $1 \leq k \leq 9$ $9^3 = 729$ variables
 satVariable($x(I,J,K)$) :- row(I), col(J), val(K).

%%%%%%%% 2. Clause generation for the SAT solver: =====

```
writeClauses :-
    filledInputValues, ..... % for each filled-in value of the input, add a unit clause
    eachIJexactlyOneK, ..... % each square  $IJ$  gets exactly one value  $K$ 
    eachJKexactlyOneI, ..... % each column  $J$  gets each value  $K$  in exactly one row  $I$ 
    eachIKexactlyOneJ, ..... % each row  $I$  gets each value  $K$  in exactly one column  $J$ 
    eachBlockEachKexactlyOnce, % each  $3 \times 3$  block gets each value  $K$  exactly once.
    true, !, ..... % this way you can comment out ANY previous line of writeClauses
writeClauses :- told, nl, write('writeClauses failed!'), nl, nl, halt.
```


Sudoku

Per cada valor emplenat de l'entrada, s'afegeix una clàusula unitària:

- 1 es carrega el sudoku (amb `entrada/1`)
- 2 es troba el valor K de la fila I, columna J (amb `nth1/3`)
- 3 si està emplenat (amb `integer/1`), s'afegeix una clàusula unitària

```

filledInputValues :-
    entrada(Sud),
    nth1(I,Sud,Row), nth1(J,Row,K),
    integer(K), writeOneClause([ x(I,J,K) ]),
    fail.
filledInputValues.
  
```

Sudoku

```

%% The Prolog predicate findall(X, Cond, L) means: "L = { X | Cond }"
eachIJexactlyOneK :- row(I), col(J), findall(x(I,J,K), val(K), Lits), exactly(1,Lits), fail.
eachIJexactlyOneK.

%% this generates, for example, exactly(1, [x(1,1,1), x(1,1,2), ... x(1,1,9)])

eachJKexactlyOneI :- col(J), val(K), findall(x(I,J,K), row(I), Lits), exactly(1,Lits), fail.
eachJKexactlyOneI.

eachIKexactlyOneJ :- row(I), val(K), findall(x(I,J,K), col(J), Lits), exactly(1,Lits), fail.
eachIKexactlyOneJ.

eachBlockEachKexactlyOnce :- blockID(Iid,Jid),
    .....val(K), findall(x(I,J,K), squareOfBlock(Iid,Jid,I,J), Lits), exactly(1,Lits), fail.
eachBlockEachKexactlyOnce.

```

Sudoku

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3. DisplaySol: show the solution. Here M contains the literals that are true in the model:

% displaySol(M):-nl,write(M),nl,nl,fail.
displaySol(M):-nl,row(I),nl,line(I),col(J),space(J),member(x(I,J,K),M),write(K),write(' '),fail.
displaySol(_):-nl,nl.

line(I):-member(I,[4,7]),nl,!.
line(_).
space(J):-member(J,[4,7]),write(' '),!.
space(_).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

Codificació en SAT

1 Generació de clàusules

2 Predicats útils

3 Codificació de sudokus

4 Llibreria adjunta

5 Exercicis

Ús de la llibreria

Des de l'interpret de Prolog, la generació de clàusules s'inicia amb la consulta

```
?- main.
```

o bé, si l'entrada es pot parametritzar,

```
?- main(<fitxer d'entrada>).
```

Per exemple, en el problema dels sudokus:

```
?- main(miSudokuExampleA).
```

Per veure les clàusules generades en format simbòlic, canvieu al fitxer `symbolicOutput(0)` per `symbolicOutput(1)`.

(El predicat es troba normalment a l'inici)

Predicats de generació de clàusules

- `writeOneClause(L)`: genera la clàusula donada per la llista L
- Cardinalitat. Generació de clàusules equivalents a
 - `exactly(K,L)`: exactament K literals de L són certs
 - `atMost(K,L)`: com a màxim K literals de L són certs
 - `atLeast(K,L)`: com a mínim K literals de L són certs
- Equivalència. Genera clàusules que estableixen que la variable V equival a...
 - `expressOr(V,L)`: ...la disjunció dels literals de L
 - `expressAnd(V,L)`: ...la conjunció dels literals de L

Predicat exactly

Està definit com:

```
exactly(K,Lits):-
    symbolicOutput(1),
    write(exactly(K,Lits)), nl, !.

exactly(K,Lits):-                % cas general
    atLeast(K,Lits),
    atMost(K,Lits), !.
```

És a dir, amb

- `symbolicOutput(1)`, s'escriu la restricció lògica que estem afegint com a entrada del *SAT solver* i
- `symbolicOutput(0)`, es generen clàusules que assegurin que **almenys** `K` literals de `Lits` són certs i que **com a màxim** `K` literals de `Lits` són certs.

Predicat atMost

```
atMost(K,Lits) :-
    symbolicOutput(1),
    write(atMost(K,Lits)), nl, !.
```

```
atMost(K,Lits) :-
    negateAll(Lits,NLits),
    K1 is K + 1,
    subsetOfSize(K1,NLits,Clause),
    writeOneClause(Claue), fail.
```

```
atMost(_,_).
```

Es basa en la propietat:

- Com a màxim K literals són certs si i només si en cada subconjunt de $K+1$ literals, almenys un literal és fals.

Predicat atLeast

```
atLeast(K,Lits) :-
    symbolicOutput(1),
    write(atMost(K,Lits)), nl, !.
```

```
atLeast(K,Lits) :-
    length(Lits,N),
    K1 is N - K + 1,
    subsetOfSize(K1,Lits,Clause),
    writeOneClause(Clause), fail.
```

```
atLeast(_,_).
```

Es basa en la propietat:

- Almenys K literals d'un total de N són certs si i només si en cada subconjunt de $N-K+1$ literals, almenys un literal és cert.

Codificació en SAT

1 Generació de clàusules

2 Predicats útils

3 Codificació de sudokus

4 Llibreria adjunta

5 Exercicis

Roads

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% The government of a country with very few but very large roads needs
%% to decide where to place gas stations. It has been decided that if
%% a road connects cities C1 and C2, a gas station should be placed in
%% C1, C2, or both. The government budget limits the number of
%% stations that can be built (maxStations).
%% Complete the following program to do this.
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%%%%%%%% begin input example roads1 %%%%%%%%%%
%%
%% numCities(20).
%% maxStations(14).
%%
%% road(18,19).
%% road(6,18).
%% road(1,12).
%% road(11,19).
%% road(3,15).

```

Roads

```
%%%%%%%%% Some helpful definitions to make the code cleaner: =====

city(C) :- numCities(N), between(1,N,C).
```

```
%%%%%%%%% 1. SAT Variables: =====
```

```
satVariable( install(C) ) :- city(C). %   install(C) means
|                                     % "a gas station is installed in city C"
```

```
%%%%%%%%% 2. Clause generation for the SAT solver: =====
```

```
writeClauses :-
|   .... %% Complete this!
|   true,!.
writeClauses :- told, nl, write('writeClauses failed!'), nl, nl, halt.
```

```
%%%%%%%%% 3. DisplaySol: show the solution. Here M contains the literals
|   % that are true in the model:
```

```
% displaySol(M) :- nl, write(M), nl, nl, fail.
displaySol(M) :-
|   .... %% Complete this!
```

Olympiad

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% For the next Catalan Computer Science Olympiad, we want to make the
%% best possible team by selecting a set of students. For each student,
%% we have the following information:
%%
%% 1.- Her ID
%% 2.- The list of other students which she considers friends. Note that
%%      S1 could consider S2 as a friend, but the converse is not
%%      necessarily true.
%% 3.- The programming topic she is most expert on.
%%
%% The goal is to form a team with exactly K students such that each
%% one of them considers all other team members a friend and all
%% chosen students have different expertise topics.
%%
%% Complete the writeClauses part below to find out one possible team.
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%% begin input example olympiad1 %%%%%%%%%%
%%
%% numMembersTeam(6). %% form a team with exactly K students
%% numStudents(25).
%%
%% %info( id, list_of_students(s)he_considers_a_friend, topic )
%% info(1,[4,9,11,14,15,16,21,23,24],t6).
%% info(2,[4,9,12,15,16,19,23],t8).

```

Olympiad

%%%%%%%% Some helpful definitions to make the code cleaner: =====

```
student(S) :-      numStudents(N), between(1,N,S).
friends(S1,S2) :- student(S1), student(S2), info(S1,L,_), member(S2,L).
expertise(S,E) :- student(S), info(S,_,E).
```

%%%%%%%% 1. SAT Variables: =====

```
satVariable( chosen(S) ) :- student(S).    % chosen(S) means "student S has been chosen"
```

%%%%%%%% 2. Clause generation for the SAT solver: =====

```
writeClauses :-
|   .... %% Complete this!
|   true,!.
writeClauses :- told, nl, write('writeClauses failed!'), nl, nl, halt.
```

%%%%%%%% 3. DisplaySol: show the solution. Here M contains the literals that are true in the model:

```
% displaySol(M) :- nl, write(M), nl, nl, fail.
displaySol(M) :-
|   write('Team: '), nl,
|   student(S), member(chosen(S), M), expertise(S,E),
|   write(S-E), nl, fail.
displaySol(_) :- nl.
```